Registro de Decisões de Transformação (RDT)

Autor: Mauricio Matias **Data:** 30/10/2026 **Versão do pipeline:** Semana 4 (etapas 01 a 05)

Repositório: https://github.com/mauriciomatias-stack/lab-etl-varejo-mercatto

Escala dos dados: `ESCALA=0.2` (440.192 linhas nas quatro fontes, contando as 21 da tabela `category`).

Cada ficha traz evidência, alternativas, decisão, o que se perde, registros afetados e asserção associada. Resultado da execução: etapa `01_extracao` com 31 asserções e 0 falhas; etapa `02_transformacao` com 31 asserções e 0 falhas; etapa `03_carga_dw` com 28 asserções e 0 falhas; etapa `04_abt` com 11 asserções e 0 falhas na versão correta (e 3 falhas na versão com vazamento, de propósito).

Cada ficha só conta se tiver "o que se perde" e "registros afetados".

Ficha RDT-01

Decisão: carga completa (e não incremental) das quatro fontes para a camada `stg`.

Etapa do pipeline: 01_extracao · `extrai_mysql`, `extrai_postgres`, `extrai_sac`, `extrai_avaliacoes`

O que o dado mostrava:

- Nenhuma das tabelas de origem tem coluna de data de alteração. O legado `PEDIDOS` tem `DT_PEDIDO` e `DT_ENTREGA`; o e-commerce `orders` tem `placed_at` e `delivered_at`; não há `atualizado_em` em nenhuma.
- O status e a data de entrega de um pedido mudam depois da carga, sem deixar rastro: 2.565 pedidos do legado (`EM TRANSITO` 1.560 + `SEPARACAO` 1.005; 9,87% de 26.000) e 7.298 do e-commerce (`shipped` 4.504 + `processing` 2.794; 13,03% de 56.000) ainda estão em andamento. Somam 9.863 pedidos, 12,03% dos 82.000.
- O SAC é exportação manual mensal concatenada: 19.228 linhas para 19.000 protocolos distintos (228 linhas repetidas, 1,19%). Não há chave única confiável.
- O volume é pequeno o bastante para recarregar inteiro: 440.192 linhas.

Alternativas consideradas:

1. Incremental por data (carregar só pedidos com `DT_PEDIDO`/`placed_at` posterior à última carga). Descartada: a data do pedido não muda, então um pedido já carregado nunca seria revisitado, e os 9.863 pedidos em andamento ficariam com o status e a data de entrega da primeira carga.
2. Incremental pela maior chave primária (só ids novos). Descartada: mesmo defeito, e o SAC nem tem chave primária utilizável.
3. Carga completa, recriando as tabelas `stg` a cada execução. **Escolhida.**
4. Carga completa guardando um histórico de cada execução. Descartada: a pergunta analítica (recompra até 28/09/2025) usa o estado dos pedidos no momento da extração, e ninguém consumiria o histórico; só multiplicaria o volume.

Decisão: alternativa 3 (`if_exists="replace"` na primeira gravação de cada tabela).

Justificativa: o enunciado exige que o pipeline rode do zero e reproduza o mesmo resultado. Com origens somente leitura, sem data de alteração e com volume pequeno, recarregar tudo é a única estratégia que garante que a `stg` espelha o estado atual da origem. Para a carga incremental valer a pena seria preciso uma coluna de alteração confiável, que não existe.

O que se perde:

- O histórico de mudanças: o estado anterior de um pedido é sobrescrito a cada carga, então não é possível saber quando um status mudou.
- Eficiência: reprocessa 100% das linhas mesmo quando quase nada mudou. Na escala 1.0 (cinco vezes maior) o custo cresce na mesma proporção.
- A coluna `_extraido_em` muda a cada execução. É só metadado, e deve ser ignorada ao comparar duas execuções.
- Nenhum valor de dado se perde: as somas monetárias conferem com a origem.

Registros afetados: 440.192 linhas (100% das fontes) são recarregadas a cada execução: legado 115.157, e-commerce 273.807, SAC 19.228 e avaliações 32.000. Dessas, 9.863 pedidos (12,03%) podem mudar de estado entre uma carga e outra.

Asserção associada: etapa `01_extracao`, 31 asserções, 0 falhas. Para cada tabela, "linhas lidas x origem" e "linhas na stg x origem" (por exemplo `PEDIDOS` 26.000 = 26.000 e `shop.order_item` 198.386 = 198.386); soma de `VALOR_TOTAL` (60.708.429,93) e de `total_amount` (146.752.362,13) dentro de ±R$ 0,01 da origem; textos com acento corrompido = 0 nas quatro tabelas do legado; SAC 19.228 = 19.228; avaliações 32.000 = 32.000, com `review_id` sem nulos e sem duplicados.

Ficha RDT-02 · Identidade do cliente entre os dois sistemas

Decisão: dois cadastros (legado e e-commerce) são o mesmo cliente quando têm o mesmo CPF de 11 dígitos, válido ou não.

Etapa do pipeline: 02_transformacao · `unifica_clientes` / `monta_clientes`

O que o dado mostrava:

- 2.516 CPFs aparecem nos dois sistemas (30,42% dos CPFs do legado e 16,07% dos do e-commerce).
- Em 2.592 pares (cliente do legado, cliente do e-commerce) com o mesmo CPF, o nome é idêntico (sem acento e caixa) em 100%.
- 232 dos 2.516 CPFs da interseção (9,22%) têm dígito verificador inválido, e o mesmo CPF inválido aparece nos dois lados para a mesma pessoa.
- Sem CPF: 900 clientes no legado (9,52%) e 1.341 no e-commerce (7,89%). E-mail e telefone em comum entre os sistemas: 0, então nada confirma um pareamento sem CPF.

Alternativas consideradas:

1. Parear por CPF exato, inclusive inválido. **Escolhida.**
2. Parear só por CPF válido. Descartada: deixaria duplicados os 232 CPFs inválidos cujos nomes coincidem, ou seja, desfaria pareamentos que os dados confirmam.
3. CPF e, para quem não tem, nome normalizado + cidade + UF. Descartada: sem e-mail nem telefone em comum não há como conferir o pareamento, e homônimos seriam fundidos.

Justificativa: fundir dois clientes distintos é pior que manter um cliente em duas linhas. A fusão errada contamina o histórico de compra, que é a base do alvo de recompra; a duplicata só reduz o histórico de um cliente. Como o nome coincide em 100% dos 2.592 pares, inclusive com CPF inválido, o dígito verificador não separa pessoas diferentes nestes dados.

O que se perde:

- Cadastros sem CPF nunca são pareados: até 2.241 (900 + 1.341) podem ser a mesma pessoa em duas linhas.
- CPF com menos de 11 dígitos é tratado como ausente (`cpf_normalizado` nulo).
- Nome e cidade são gravados em MAIÚSCULAS sem acento (`CUIABA`); o acento original fica só na `stg`.
- A regra confia que um CPF digitado errado de forma idêntica em duas pessoas não ocorre; nos dados, 0 pares contrariam isso.

Registros afetados: 26.450 cadastros de origem (9.450 do legado + 17.000 do e-commerce) viram 23.655 linhas em `dim_cliente`, 2.795 a menos (10,57%). Dessas linhas: 2.516 `AMBOS` (10,64%), 6.655 `LEGADO` e 14.484 `ECOM`. Sem CPF, sem pareamento: 2.241 (9,47% das linhas), 900 do legado e 1.341 do e-commerce. CPFs de 11 dígitos: 21.414, dos quais 1.977 (9,23%) com dígito verificador inválido.

Asserção associada: etapa `02_transformacao` (31 asserções no total, 0 falhas; as deste ponto são): `dim_cliente` com 23.655 linhas, igual à contagem independente em SQL (uma por CPF de 11 dígitos mais uma por cadastro sem CPF); 0 CPFs duplicados entre válidos e entre todos; `origem_unificacao` dentro de {LEGADO, ECOM, AMBOS}; `AMBOS` = 2.516, igual à interseção medida na semana 1.

Ficha RDT-03 · Duplicatas internas do legado

Decisão: cadastros do legado com o mesmo CPF são fundidos em uma única linha de `dim_cliente`.

Etapa do pipeline: 02_transformacao · `unifica_clientes` / `monta_clientes`

O que o dado mostrava: 279 CPFs aparecem em mais de um cliente do legado (558 linhas; 5,90% dos 9.450); 9.450 clientes para 9.000 e-mails distintos (450 linhas a mais, 4,76%). O e-commerce não tem CPF repetido.

Alternativas consideradas:

1. Manter as duas linhas e só pareá-las com o e-commerce. Descartada: o mesmo cliente teria dois `cliente_sk`, e a frequência de compra seria dividida entre eles.
2. Fundir por CPF, com a mesma regra do RDT-02. **Escolhida.**
3. Fundir também duplicatas sem CPF por nome e e-mail. Descartada: 450 e-mails repetidos não provam a mesma pessoa, e não há como conferir.

Justificativa: é a mesma regra do RDT-02, aplicada dentro do legado, e os nomes dos pares com CPF igual coincidem. Os pedidos dos dois `ID_CLIENTE` passam a apontar para o mesmo `cliente_sk` pela tabela `dw.ponte_cliente`, porque `dim_cliente` guarda um só `id_legado`.

O que se perde: `dim_cliente.id_legado` guarda apenas o `ID_CLIENTE` do cadastro mais recente; o outro só existe em `ponte_cliente`. Nome, cidade e UF vêm do cadastro mais recente (valor não nulo); `dt_primeiro_cadastro` é a menor data.

Registros afetados: 279 cadastros do legado fundidos em outro cadastro com o mesmo CPF (2,95% dos 9.450): 203 CPFs repetidos só no legado e 76 que também existem no e-commerce. Os 558 cadastros de origem desses CPFs ocupam 279 linhas em `dim_cliente`.

Asserção associada: `ponte_cliente` com 26.450 linhas, uma por cadastro de origem (9.450 + 17.000); `dim_cliente` com 0 CPFs duplicados; `linhas_legado_fundidas` = 279.

Ficha RDT-04 · Conformação de status

Decisão: `status_conformado = CANCELADO` para os três cancelamentos de origem, com `motivo_cancelamento` preenchido só quando a origem distingue (`CLIENTE`, `PAGAMENTO_RECUSADO`).

Etapa do pipeline: 02_transformacao · `conforma_status` (`MAPA_STATUS`)

O que o dado mostrava:

- Legado: `CANCELADO` 3.161 (12,16% de 26.000); a origem não diz o motivo.
- E-commerce: `canceled` 4.517 (8,07% de 56.000) e `payment_declined` 2.832 (5,06%).
- Demais status: `ENTREGUE` 20.274 × `delivered` 41.353; `EM TRANSITO` 1.560 × `shipped` 4.504; `SEPARACAO` 1.005 × `processing` 2.794.

Alternativas consideradas:

1. `payment_declined` → `CANCELADO`, sem motivo. Descartada: perde a distinção justamente onde ela existe.
2. `CANCELADO_CLIENTE` e `CANCELADO_PAGAMENTO` como status separados. Descartada: os 3.161 cancelados do legado não cabem em nenhum dos dois, e qualquer contagem de "cancelados" teria de somar valores.
3. `CANCELADO` com coluna de motivo, nula quando a origem não distingue. **Escolhida.**

Justificativa: `CANCELADO` é o único nível comparável entre os dois sistemas, e a coluna `motivo_cancelamento` da `dim_status` foi desenhada para guardar a distinção quando ela existe. Para a recompra, uma recusa de pagamento também não é compra concluída, então todos os cancelados saem da contagem de compras.

O que se perde:

- Nos 3.161 cancelados do legado o motivo fica nulo: não dá para saber quantos foram recusa de pagamento. Qualquer análise por motivo só vale para o e-commerce.
- Hipóteses assumidas sem confirmação no dicionário: `processing` equivale a `SEPARACAO`, e `canceled` é cancelamento pelo cliente.

Registros afetados: 10.510 pedidos cancelados (12,82% dos 82.000): 3.161 sem motivo, 4.517 com motivo `CLIENTE` e 2.832 com motivo `PAGAMENTO_RECUSADO`. Os outros 71.490 mudam só de nome (por exemplo `EM TRANSITO` → `EM_TRANSITO`). Total mapeado: 82.000 (100%).

Asserção associada: etapa `02_transformacao`: 0 pedidos sem mapeamento; pedidos mapeados = total na `stg` (82.000); `dim_status` com 9 linhas.

Canais (mesma etapa): `LOJA` → `FISICO`; `TELEVENDAS` → `TELEVENDAS`; `SITE`, `web` e `app` → `ONLINE_PROPRIO`; `marketplace` → `MARKETPLACE`. Perde-se a diferença entre site, web e app dentro de `ONLINE_PROPRIO`, que continua em `canal_origem`.

Ficha RDT-05 · Semântica de `VALOR_TOTAL` e `total_amount`

Decisão: `valor_mercadoria` = Σ(quantidade × preço), sem frete e sem desconto; `valor_desconto` = Σ desconto; `valor_total` = mercadoria + frete − desconto, igual nos dois sistemas.

Etapa do pipeline: 02_transformacao · `monta_pedidos` / `conforma_valores`

O que o dado mostrava (reconciliação com a soma dos itens, tolerância R$ 0,02; o dicionário não define os campos):

- Legado: Σ(qtd × preço − desconto) + frete bate em 25.239 de 26.000 pedidos (97,07%); tratando o desconto `-1` como zero, bate em 26.000 (100%). O `VALOR_TOTAL` **inclui** o frete.
- E-commerce: Σ(qtd × preço − desconto) bate em 56.000 de 56.000 (100%). O `total_amount` **não inclui** o frete.

Alternativas consideradas:

1. Carregar `VALOR_TOTAL` e `total_amount` como vieram. Descartada: medem coisas diferentes, e somar os dois seria comparar valores com e sem frete.
2. Definir o total sem frete nos dois sistemas. Descartada: o DDL define `valor_total = mercadoria + frete − desconto`.
3. Reconstruir as medidas a partir dos itens e do frete, com a fórmula do DDL. **Escolhida.**

Justificativa: só a reconstrução produz uma medida com o mesmo significado nos dois sistemas, e ela é verificável: bate com o total da origem em 100% dos pedidos (legado direto; e-commerce depois de retirar o frete).

O que se perde:

- No e-commerce, `valor_total` do DW é maior que o `total_amount` da origem pelo valor do frete. A reconciliação financeira da semana 3 deve comparar `valor_total − valor_frete` com `total_amount`.
- No legado, a diferença nos 761 pedidos com desconto `-1` está na correção do RDT-06.

Registros afetados: 82.000 pedidos (100%) têm as quatro medidas recalculadas; nos 56.000 do e-commerce o `valor_total` passa a incluir o frete.

Asserção associada: etapa `02_transformacao`: divergência entre total conformado e total da origem = 0 nos dois sistemas (±0,02); `valor_total = mercadoria + frete − desconto` em 100% das linhas; mercadoria − desconto = soma dos itens (±0,01) por pedido.

Ficha RDT-06 · Desconto `-1`, frete negativo e preços extremos

Decisão: (a) desconto `-1` do legado é sentinela e vira 0; (b) frete negativo é mantido como veio; (c) preços 10× acima da mediana do produto são só medidos (a medição deu 0). Os itens órfãos têm ficha própria (RDT-10).

Etapa do pipeline: 02_transformacao · `monta_pedidos`

O que o dado mostrava:

- Desconto `-1`: 770 itens do legado (0,97% de 79.107), em 761 pedidos (2,93% de 26.000). O e-commerce não tem desconto negativo (mínimo 0,00). Tratado como zero, o total reconcilia em 100%.
- Frete negativo: 86 pedidos do legado (0,33%) e 152 do e-commerce (0,27%). Não impede a reconciliação do total.
- Preços 10× acima da mediana do próprio produto: 0 itens, então não há o que corrigir nem excluir.

Alternativas consideradas:

- Desconto `-1`: (a) tratar como zero, **escolhida**; (b) manter o `-1`, que somaria R$ 1 ao item, mas não reconcilia com o total da origem.
- Frete negativo: (a) zerar; (b) manter, **escolhida**; (c) manter e marcar. Zerar mudaria o total de pedidos que a origem já reconcilia, e o DDL não tem coluna de marcação.
- Preço 10×: corrigir ou excluir. Descartada: não há como distinguir erro de digitação de preço alto legítimo, e mexer quebraria a reconciliação com o total da origem. A medição não achou nenhum caso.

Justificativa: o `-1` é marca de "sem desconto", não um valor; o frete negativo reconcilia com o total e pode ser estorno, então a origem é preservada.

O que se perde:

- O frete negativo continua no DW; qualquer média de frete precisa decidir se o inclui.
- Nenhum preço 10× acima da mediana do produto foi encontrado, então o critério não alterou nenhum valor.

Registros afetados: 770 itens e 761 pedidos (desconto); 238 pedidos (frete negativo, 0,29% dos 82.000); 0 itens com preço 10× acima da mediana.

Asserção associada: itens sem produto = 0; divergência entre o total conformado e o da origem = 0 nos dois sistemas.

Ficha RDT-07 · Mapeamento de categorias

Decisão: o texto livre do legado é mapeado para a taxonomia do e-commerce; `categoria_origem` guarda o valor bruto. Valores sem informação viram `NAO_CLASSIFICADO`; o único com dois ramos vira `AMBIGUO`. Produtos dos dois sistemas não são pareados.

Etapa do pipeline: 02_transformacao · `conforma_categorias` (`MAPA_CATEGORIA`)

O que o dado mostrava: 27 valores de texto livre no legado, 22 produtos com categoria vazia (3,67% de 600), `DIVERSOS` (28) e `OUTROS` (24); taxonomia do e-commerce com 21 nós, dos quais os produtos usam 15. Casos ambíguos: `informatica e games` (23 produtos) cobre dois ramos; `TV E SOM` (21) cobre dois filhos de Áudio e TV. Variantes de grafia do mesmo conceito: `eletrodomesticos` (34), `Eletro` (30), `Eletro-domesticos` (19). O legado não tem SKU.

Alternativas consideradas:

1. Mapear `informatica e games` para Informática (ou para Games). Descartada: classificaria 23 produtos num ramo sem base.
2. Criar uma categoria composta "Informática e Games". Descartada: sai da taxonomia controlada.
3. Marcar como `AMBIGUO`, com o bruto preservado. **Escolhida.**
4. Para `DIVERSOS`, `OUTROS` e vazio: `NAO_CLASSIFICADO`, e não palpitar por descrição. **Escolhida.**
5. Parear produtos do legado com os do e-commerce por descrição. Descartada: as descrições não coincidem e não há chave comum.

Justificativa: errar a categoria contamina análises por categoria; admitir que não se sabe não contamina. O valor bruto continua em `categoria_origem`, então a decisão pode ser revista sem reextrair.

O que se perde:

- 74 produtos (`DIVERSOS` 28, `OUTROS` 24, vazio 22; 12,33% de 600) não têm categoria útil, e 23 (3,83%) ficam `AMBIGUO`.
- `TV E SOM`, `AUDIO/VIDEO` e `GAMES E CONSOLES` param no primeiro nível: o segundo nível fica nulo.
- Como não há pareamento, o mesmo produto físico pode estar duas vezes em `dim_produto` (600 do legado + 2.400 do e-commerce = 3.000 linhas).

Registros afetados: 97 dos 600 produtos do legado (16,17%) sem categoria conformada; 0 nos 2.400 do e-commerce.

Asserção associada: valores do legado sem regra = 0; `dim_produto` = 3.000 linhas; `categoria_origem` do legado idêntica ao bruto em 600 de 600.

Ficha RDT-08 · Tipos de ausência

Decisão: cada ausência é classificada antes de qualquer tratamento. Nulo estrutural permanece nulo; sentinela `1900-01-01` e data de entrega anterior ao pedido viram NULL; nenhuma é preenchida com média ou mediana.

Etapa do pipeline: 02_transformacao · `monta_pedidos` e `trata_datas_e_ausencias`

O que o dado mostrava:

- Nulo estrutural (entrega ainda não aconteceu): 5.726 pedidos no legado (22,02%) e 14.647 no e-commerce (26,16%), todos em pedidos não entregues.
- Sentinela `1900-01-01`: 222 pedidos entregues do legado (1,09% dos entregues). No e-commerce, a menor data de entrega é 2023-01-02, sem sentinela.
- Data impossível (entrega antes do pedido): 140 no legado (0,69% dos entregues) e 343 no e-commerce (0,83%).
- SAC: `data_fechamento` vazia em 1.168 atendimentos (6,07%).

Alternativas consideradas:

- Sentinela: (a) manter `1900-01-01`, que geraria prazos de mais de 120 anos; (b) NULL, **escolhida**; (c) imputar pelo prazo médio, que inventa um dado.
- Entrega antes do pedido: (a) trocar as datas; (b) manter e sinalizar; (c) NULL, **escolhida**. Não se sabe qual das duas datas está errada, e o DDL não tem coluna de sinalização.
- SAC sem fechamento: manter NULL; a origem não permite distinguir atendimento em aberto de dado perdido.

Justificativa: cada tipo de ausência diz uma coisa diferente: "ainda não aconteceu" é informação, a sentinela é um dado desconhecido, e a data impossível é um dado errado. Em todos os casos NULL é o único valor que não afirma algo falso.

O que se perde: 705 pedidos entregues (222 + 140 + 343) ficam sem data de entrega e sem `dias_entrega`; a data original só permanece na `stg`. Nos 1.168 atendimentos não dá para calcular o tempo de resolução a partir das datas.

Registros afetados: 705 pedidos (1,14% dos 61.627 entregues): 222 sentinelas do legado, 140 do legado e 343 do e-commerce com entrega antes do pedido; 1.168 atendimentos do SAC (6,07%). Nulo estrutural preservado: 5.726 no legado e 14.647 no e-commerce. Entregas válidas: 19.912 no legado e 41.010 no e-commerce.

Asserção associada: 0 entregas antes do pedido e 0 datas de 1900 no DW; `dias_entrega` negativo = 0; sentinelas tratadas = contagem independente na `stg`; pedidos entregues sem data = sentinelas + impossíveis; pedido não entregue com data = 0.

Ficha RDT-09 · Padronização de fuso horário

Decisão: todas as datas do DW são datas de calendário no fuso `America/Sao_Paulo`. O `DATETIME` do legado, sem fuso, é tomado como horário local; o `TIMESTAMPTZ` do e-commerce é convertido de UTC para esse fuso.

Etapa do pipeline: 02_transformacao · `monta_pedidos`, `monta_clientes`

O que o dado mostrava: `DT_PEDIDO` do legado é DATETIME sem fuso; `placed_at` e `delivered_at` do e-commerce são TIMESTAMPTZ em UTC; `created_at` dos clientes do e-commerce aparece sempre em `03:00:00+00:00`, ou seja, meia-noite em Brasília guardada em UTC; no NDJSON, 6.427 de 32.000 `submitted_at` (20,08%) vêm sem fuso; no SAC as datas são `dd/mm/aaaa`, sem hora.

Alternativas consideradas:

1. Padronizar em UTC. Descartada: moveria para o dia anterior ou seguinte as compras feitas à noite no horário local, e o legado já está em hora local.
2. Padronizar em `America/Sao_Paulo`. **Escolhida.**
3. Ignorar o fuso e usar a data como veio. Descartada: o mesmo instante teria datas diferentes nos dois sistemas.

Justificativa: o legado e o SAC são locais e não têm fuso para converter; o e-commerce é o único que declara UTC, e o `03:00` dos cadastros mostra que a hora local é Brasília. É o mesmo calendário que o cliente viveu.

O que se perde: a hora do dia (o grão do DW é o dia). Nas avaliações sem fuso (20,08%), assume-se o horário local, o que pode errar por até um dia se a origem estiver em UTC. Cuiabá (UTC−4) difere de Brasília em uma hora; a diferença só muda o dia de eventos entre 23h e meia-noite.

Registros afetados: 7.095 dos 56.000 pedidos do e-commerce (12,67%) mudam de dia ao converter de UTC para o fuso local; 6.427 avaliações sem fuso (na carga da semana 3).

Asserção associada: 0 datas fora do calendário `dim_tempo` (integridade referencial na carga de `fato_pedido`).

Ficha RDT-10 · Itens órfãos

Decisão: itens cujo pedido não existe na origem não entram em `fato_item_pedido`. Eles vão para a tabela de quarentena `qa.item_orfao`, com o valor da linha, e ficam fora de qualquer soma do DW.

Etapa do pipeline: 02_transformacao (`monta_pedidos`) e 03_carga_dw

O que o dado mostrava:

- Legado: 1.169 itens (1,48% dos 79.107) apontam para 1.100 pedidos que não existem em `PEDIDOS`. O maior `ID_PEDIDO` em `ITENS_PEDIDO` é 35.997, e o maior em `PEDIDOS` é 26.000.
- E-commerce: 2.931 itens (1,48% dos 198.386) apontam para 2.539 pedidos que não existem em `orders`. O maior `order_id` em `order_item` é 65.997, e o maior em `orders` é 56.000.
- Nenhum item órfão fica sem produto (0), e nenhum pedido fica sem itens (0), então o problema é só de itens sem pedido.
- As origens não têm chave estrangeira declarada, e o acesso é somente leitura: não há como corrigir na origem.

Alternativas consideradas:

1. Descartar os órfãos sem registro. Descartada: some dinheiro sem deixar rastro, e a diferença apareceria só na reconciliação.
2. Criar um pedido artificial para cada pedido inexistente. Descartada: inventaria pedido, cliente e data que a origem não tem, e contaminaria a ABT com compras que podem não ter existido.
3. Carregar o item com `pedido_sk` nulo. Descartada: o DDL declara `pedido_sk NOT NULL` em `fato_item_pedido`.
4. Manter fora das fatos e guardar em quarentena (`qa.item_orfao`). **Escolhida.**

Justificativa: sem pedido, o item não tem cliente nem data, e a pergunta analítica (recompra por cliente) não consegue usá-lo. A quarentena preserva a evidência, para que alguém com acesso à origem possa investigar, sem deixar o valor entrar em nenhuma soma do DW.

O que se perde: o valor desses itens não entra em nenhuma medida do DW, nem na reconciliação financeira. Se os pedidos faltantes existirem em outro sistema, essas vendas ficam fora da análise. Valor bruto (quantidade × preço) dos itens em quarentena: **R$ 556.140,00**, sendo R$ 116.783,10 do legado (1.100 pedidos) e R$ 439.356,90 do e-commerce (2.539 pedidos), cerca de 0,27% do valor total de R$ 209.654.657,47 do DW.

Registros afetados: 4.100 itens (1,48% dos 277.493): 1.169 do legado e 2.931 do e-commerce. Entram em `fato_item_pedido` 273.393 itens.

Asserção associada: `fato_item_pedido` = itens que têm pedido na origem (273.393, contagem independente em SQL); `qa.item_orfao` = 4.100; itens sem produto = 0.

Ficha RDT-11 · Declaração de grão e definição da reconciliação financeira

Decisão: `fato_pedido` tem o grão de **um pedido de um sistema de origem**, identificado por `(sistema_origem, id_origem)`, com `id_origem` no formato `LEG-123` ou `ECOM-456`. `fato_item_pedido` tem o grão de **uma linha de item de um pedido**. `qtd_itens` é a soma das unidades, não o número de linhas.

Etapa do pipeline: 03_carga_dw

O que o dado mostrava:

- 26.000 números de pedido existem nos dois sistemas (os ids do legado de 1 a 26.000 também existem no e-commerce). Sem o sistema, o número sozinho não identifica o pedido.
- São 26.000 pedidos do legado e 56.000 do e-commerce, 82.000 no total. A única ponte entre os dois sistemas é o CPF do cliente (e-mail e telefone em comum: 0). Para os 26.000 números que colidem, a consulta de verificação no DW (junção legado × e-commerce pelo número) mostra que, dos 26.000 números em comum, **0** têm o mesmo cliente, **25** (0,10%) têm a mesma data (coincidência) e **0** têm o mesmo valor total. São, portanto, pedidos distintos que apenas compartilham numeração, e fundi-los pelo número misturaria compras de clientes diferentes.
- O SAC e as avaliações referenciam o pedido por `(origem_sistema, id)` ou por `LEG-n` / `ECOM-n`, o que sustenta a chave composta.

Alternativas consideradas:

1. Usar só o número do pedido como chave. Descartada: 26.000 números colidem.
2. Unificar os pedidos que têm o mesmo número nos dois sistemas. Descartada: não há chave que ateste que sejam o mesmo pedido (ver a comparação acima), e fundir somaria valores de compras distintas.
3. Chave composta `(sistema_origem, id_origem)`, com `id_origem` prefixado. **Escolhida.**
4. Para `qtd_itens`: contar linhas em vez de unidades. Descartada: um item de 3 unidades valeria o mesmo que um de 1, e o valor do pedido cresce com as unidades.

Reconciliação financeira (exigência de 0,1%): a soma de `valor_total` do DW, por sistema, é comparada com o total calculado na origem **pela mesma definição** do DW (RDT-05):

- Legado: `SUM(VALOR_TOTAL)`, que já inclui o frete.

- E-commerce: `SUM(total_amount) + SUM(freight_amount)`, porque o `total_amount` da origem não inclui o frete.

Comparar o `valor_total` do e-commerce direto com `SUM(total_amount)` daria uma diferença igual ao frete (cerca de 1,5%), que é um efeito da definição, e não uma perda da carga.

Justificativa: a chave composta é a única que reproduz o que as fontes de arquivo já usam para apontar para o pedido. A reconciliação com a mesma definição prova que a carga não perdeu nem inventou valor.

O que se perde: a chave é mais longa e todo cruzamento precisa levar o sistema junto; pedidos de mesmo número nos dois sistemas nunca são comparados como "o mesmo pedido". A reconciliação compara o DW com a origem recalculada, e não com o `total_amount` bruto do e-commerce, e isso fica explícito aqui.

Registros afetados: 82.000 pedidos (100%) e 273.393 itens (100% dos que têm pedido). Reconciliação (saída de `03_carga_dw.py`): legado R$ 60.708.429,93 na origem e R$ 60.708.429,93 no DW; e-commerce R$ 148.946.227,54 na origem (`total_amount` + frete) e R$ 148.946.227,54 no DW; total R$ 209.654.657,47 nos dois lados. A diferença é R$ 0,00 nos dois sistemas (erro relativo da ordem de 1e-15, só arredondamento de ponto flutuante), contra a tolerância de 0,1%.

Asserção associada: `fato_pedido` = 82.000 linhas, sem duplicatas em `(sistema_origem, id_origem)`; reconciliação financeira por sistema dentro de 0,1%.

Ficha RDT-12 · Pedidos sem cliente correspondente

Decisão: nenhum pedido fica sem cliente em `dim_cliente`, e nenhum registro-sentinela "cliente desconhecido" é criado. A regra é verificada a cada carga, e qualquer pedido sem cliente iria para quarentena, e não para `fato_pedido`.

Etapa do pipeline: 03_carga_dw

O que o dado mostrava: na semana 1, 0 pedidos do legado e 0 do e-commerce apontavam para um cliente inexistente (`ID_CLIENTE` e `customer_id` presentes nas tabelas de clientes), mesmo sem chave estrangeira declarada nas origens. Todos os 26.450 cadastros de origem têm uma linha em `ponte_cliente`, e todos os 82.000 pedidos encontram o seu `cliente_sk`.

Alternativas consideradas:

1. Criar a linha "CLIENTE DESCONHECIDO" em `dim_cliente` para receber esses pedidos. Descartada: o pedido apareceria na ABT como se fosse de um cliente real, e todos os pedidos órfãos se fundiriam em um só.
2. Carregar com `cliente_sk` nulo. Descartada: o DDL declara `cliente_sk NOT NULL`.
3. Excluir o pedido da carga e registrar em quarentena. **Escolhida como regra**, hoje sem nenhum caso.

Justificativa: o problema não ocorre nos dados atuais, mas a origem não tem chave estrangeira, então pode ocorrer numa carga futura. Prefere-se uma regra que falhe de forma visível (asserção) a um valor artificial que esconda o problema.

O que se perde: nada hoje. Se algum dia ocorrer, o pedido não entra no DW nem na ABT.

Registros afetados: 0 pedidos (0% dos 82.000).

Asserção associada: pedidos sem cliente, canal ou status = 0 (já ativa na etapa 02) e pedidos sem cliente em `fato_pedido` = 0 na etapa 03.

Ficha RDT-13 · Atendimentos (SAC) e avaliações: ligação ao pedido, ao cliente e ao produto

Decisão: todos os atendimentos e todas as avaliações são carregados (`fato_atendimento` e `fato_avaliacao`), sem descartar linha por falta de par. O pedido é ligado pela chave composta `(origem_sistema, id_pedido_origem)` no SAC e pelo prefixo `LEG-n` / `ECOM-n` nas avaliações. O **cliente do atendimento vem só do CPF**; o cliente do pedido **não** é usado como reserva. Onde o par não existe, a coluna fica NULL (`cliente_sk` em 1.229 atendimentos; `produto_sk` em 7.959 avaliações). Valores fora de domínio viram NULL, nunca são "consertados".

Etapa do pipeline: 03_carga_dw (`monta_atendimentos`, `monta_avaliacoes`)

O que o dado mostrava:

- SAC: 19.228 linhas para 19.000 protocolos; as 228 linhas a mais são duplicatas exatas. Os 19.000 atendimentos ligam a um pedido do DW pela chave composta. Sem `origem_sistema`, a ligação seria ambígua: os 7.292 números de pedido do legado também existem no e-commerce.
- Cliente do SAC: o CPF está presente em todos os atendimentos, mas só 17.771 (93,53%) encontram um cliente em `dim_cliente`; 1.229 (6,47%) não encontram. Entre os 17.771 que têm cliente pelo CPF, **apenas 1** tem o mesmo cliente do pedido ao qual o atendimento está ligado; em **17.770** o CPF e o pedido apontam para pessoas diferentes. Ou seja, a chave do pedido é válida, mas o atendimento não pertence ao cliente daquele pedido.
- Avaliações: 32.000 linhas, 100% ligam a um pedido pelo `order.ref`. O `product.sku` só existe em `dim_produto` para 24.041 (75,13%); as 7.959 do legado ficam sem produto, porque o SKU não existe no cadastro do e-commerce.

- Notas: no SAC, 584 estão fora de 1 a 5 e 1.711 são nulas; nas avaliações, 941 são nulas ou ilegíveis (o campo vem como inteiro, texto ou ausente).
- Datas: 25.573 avaliações têm fuso e 6.427 (20,08%) não têm. No SAC, 1.150 atendimentos têm `data_fechamento` vazia (já sem as duplicatas).
- Outros: 87 tempos de resolução negativos; vírgula decimal no tempo de resolução; 36 textos livres de motivo; canal com variações de caixa.

Alternativas consideradas:

1. Descartar atendimentos e avaliações sem par confiável. Descartada: some informação e esconde o problema; a ligação ao pedido e ao produto pode ser NULL sem impedir o uso do restante.
2. Para o cliente do SAC, usar o cliente do pedido (ou o CPF e, na falta, o pedido). Descartada: em 17.770 de 17.771 casos os dois discordam, então usar o pedido atribuiria o atendimento a outra pessoa e, na ABT de recompra, pareceria um sinal do cliente errado.
3. Cliente do SAC só pelo CPF, NULL quando não há par. **Escolhida.**
4. Nota fora de 1 a 5: ajustar para o limite mais próximo (clamp). Descartada: inventa um valor; vira NULL, igual à nota nula.
5. Motivo: manter os 36 textos livres. Descartada: o mesmo motivo escrito de formas diferentes fragmenta qualquer contagem. Os sufixos "OBS: CLIENTE REITEROU O CONTATO" (265 casos) são removidos e três sinônimos são fundidos (`ATRASO ENTREGA` em `ATRASO NA ENTREGA`, `CANCELAMENTO A PEDIDO DO CLIENTE` em `CANCELAMENTO CLIENTE`, `TROCA DE PRODUTO` em `TROCA`), o que dá 12 motivos.

Justificativa: a chave de pedido é válida (existe pedido com aquele `(sistema, id)`), mas não se confirma pelo cliente, e o CPF é o que identifica a pessoa. Por isso o CPF decide o cliente, e o `pedido_sk` do SAC fica como informação da fonte, sem ser usado para inferir quem é o cliente. Datas sem fuso das avaliações são tomadas como horário local (`America/Sao_Paulo`), a mesma regra do RDT-09; `data_fechamento` vazia fica NULL (atendimento em aberto ou dado perdido, e a origem não permite distinguir).

O que se perde: os 1.229 atendimentos sem cliente não podem entrar em nenhum atributo da ABT por cliente; as 7.959 avaliações sem produto não entram em análises por produto; o texto livre de motivo e a observação de reiteração de contato; a hora do dia e a distinção entre fechamento em aberto e perdido. Quem juntar o SAC ao cliente por `pedido_sk` obtém pessoas diferentes das do CPF, e isso precisa ser evitado.

Registros afetados:

- SAC: 228 duplicatas removidas (1,19% de 19.228); 1.229 sem cliente (6,47% de 19.000); 584 notas fora de domínio (3,07%) e 1.711 nulas (9,01%); 87 tempos negativos (0,46%); 1.150 sem fechamento (6,05%); 265 motivos com sufixo (1,39%).
- Avaliações: 7.959 sem produto (24,87% de 32.000); 941 sem nota utilizável (2,94%); 6.427 sem fuso (20,08%).

Asserção associada: `fato_atendimento` = 19.000 e `fato_avaliacao` = 32.000 linhas (stg = carregadas + removidas); `protocolo` e `review_id` únicos; nota fora de 1 a 5 = 0; motivos distintos = 12; integridade referencial dos dois fatos (quando preenchidos) = 0 órfãos.

Ficha RDT-14 · Desenho dos atributos da ABT e verificação de ausência de vazamento

Evidência:

- Pergunta: dado o comportamento até `2025-06-30`, o cliente compra de novo entre `2025-07-01` e `2025-09-28`? Base: 22.106 clientes com pedido até o corte; 4.801 recompram (pedido não cancelado na janela), taxa 0,217. Contando todos os status a taxa seria 0,246; só entregues, 0,190.
- Quinze atributos, todos com data de evento ≤ corte: recência, frequência, valor monetário, ticket médio, prazo médio de entrega, taxa de cancelamento, categorias distintas, canal predominante, sistemas distintos, atendimentos e nota do SAC, avaliações e nota, UF, tempo como cliente.
- `dt_primeiro_cadastro` é posterior ao corte em 3.371 dos 22.106 clientes da base (15,25%) e posterior ao primeiro pedido em 16.410 (74%). 456 pedidos anteriores ao corte foram entregues depois dele.

Alternativas consideradas:

1. Tempo como cliente a partir de `dt_primeiro_cadastro`. Descartada: usa uma data que, para 74% dos clientes, vem depois do primeiro pedido, e para 15,25% vem depois do corte.
2. Usar `origem_unificacao` e `regra_pareamento` como atributos. Descartada: dependem do cruzamento de CPF sobre a base inteira (RDT-02), logo carregam informação de depois do corte.
3. Prazo de entrega com todas as entregas dos pedidos pré-corte. Descartada: 456 entregas ocorreram depois do corte.
4. Alvo com todos os status (taxa 0,246). Descartada: um pedido cancelado não é recompra.
5. **Escolhida:** tempo como cliente pelo primeiro pedido, prazo só com entregas até o corte, atributos de origem de CPF excluídos, alvo com pedidos não cancelados.

Decisão: `abt.recompra_90d`, uma linha por cliente-base, com os quinze atributos acima, `data_corte` e o alvo `alvo_recompra_90d`. Categorias `NAO_CLASSIFICADO` e `AMBIGUO` (RDT-07) não contam em `categorias_distintas`.

Justificativa: o único ponto do pipeline que pode olhar para o futuro é o alvo. Para provar que os atributos não olham, a etapa 04 recalcula por fora do SQL (em pandas, com o filtro de data explícito) frequência, valor monetário e recência e compara linha a linha com a ABT.

O que se perde: o status do pedido na DW é o da extração, sem data de cancelamento. Um pedido feito antes do corte e cancelado depois entra como cancelado em `taxa_cancelamento` e fora de frequência e valor, o que injeta informação posterior ao corte. Não há como corrigir com os dados disponíveis; fica como limitação. Perdem-se também as informações de origem de CPF (alternativa 2).

Registros afetados: 22.106 clientes na ABT (nenhum perdido na junção: clientes sem atributos = 0); 3.371 clientes com cadastro posterior ao corte e 456 pedidos com entrega posterior ao corte tratados como descrito.

Resultado e limite: com a ABT correta (partição 70/30 estratificada, teste nos 30%), o Naive Bayes dá AUC 0,552, a Regressão Logística 0,549 e a Árvore 0,507. Isto fica abaixo da faixa que o professor indica (0,62 a 0,72). Investiguei antes de aceitar: o AUC de cada atributo isolado fica entre 0,48 e 0,54, frequência nos 90, 180 e 365 dias antes do corte dá 0,51 a 0,53, e o volume por trimestre é constante (cerca de 6.800 pedidos). Nada nos dados mostra sinal forte de recompra; não ajustei o alvo para chegar à faixa.

Asserção associada: 11 asserções, 0 falhas na versão correta: taxa do alvo entre 0,02 e 0,60; `cliente_sk` único; linhas do alvo = linhas do SQL; cardinalidade preservada; clientes sem atributos = 0; `data_corte` único; frequência, valor monetário e recência iguais ao recálculo com filtro de data; recência negativa = 0.

Ficha RDT-15 · Vazamentos introduzidos deliberadamente na versão de comparação

Evidência: a tabela `abt.recompra_90d_vazamento` usa o mesmo SQL e as mesmas colunas; muda só o que está listado abaixo. Mesmos 22.106 clientes e mesmo alvo.

Vazamento	Linha de código	O que muda	Efeito medido
1. `recencia_dias`	filtro de data trocado por `TRUE`	última compra pode cair na janela do alvo ou depois	8.615 clientes com recência negativa (mínimo −183); AUC isolado 0,478 (correta) contra 0,808 (1 − 0,192 na versão com vazamento)
2. `frequencia_pedidos`	filtro de data trocado por `TRUE`	conta os pedidos da janela do alvo	8.615 linhas diferentes da correta; média 4,29 entre quem recompra contra 2,90 entre quem não recompra (na correta, 2,86 e 2,65); AUC isolado 0,530 contra 0,712
3. `ticket_medio`	z-score com média e desvio de todas as linhas	escala calculada antes da partição treino/teste	AUC isolado inalterado (0,503); efeito pequeno e silencioso

Alternativas consideradas: usar só os vazamentos 1 e 2 (os que chamam atenção) ou incluir o 3 para mostrar o vazamento que não chama atenção. **Escolhida:** os três, porque o terceiro é o que passa na revisão visual.

Decisão: manter as duas ABTs lado a lado, rodar o mesmo fluxo Orange (Data Sampler 70/30 estratificado, três modelos, Test on test data, classe 1) e comparar.

Justificativa: a diferença entre as duas versões cabe em uma linha de SQL. Mostrar isso, e mostrar como a asserção de recálculo pega o erro, é o objetivo do experimento (relatório em `docs/Relatorio_Vazamento`).

O que se perde: nada na DW. A tabela com vazamento nunca deve ser usada para decidir; só para o experimento. O vazamento 3 não foi isolado em um experimento próprio, então não medi seu ganho de AUC separadamente.

Registros afetados: 22.106 clientes; 8.615 (38,97%) com recência negativa e frequência diferente da correta.

Asserção associada: a mesma `verifica_sem_vazamento` da RDT-14, rodada nas duas versões: 11 asserções e 0 falhas na correta; 11 asserções e 3 falhas na versão com vazamento (frequência, recência e recência negativa, 8.615 linhas). A falha é a prova de que a asserção enxerga o erro. O z-score do ticket médio não é pego por essa asserção; o que o denuncia é média 0 e desvio 1 em toda a base antes da partição.

Tabela-resumo (fichas 01 a 15)

Ficha	Decisão	Registros afetados	% da base	Informação perdida?
RDT-01	Carga completa	440.192	100%	Não se perde valor; perde-se o histórico de mudanças de status
RDT-02	Pareamento por CPF exato	2.516 pareados	10,64% das 23.655 linhas	2.241 sem CPF não pareados; acentos só na `stg`
RDT-03	Fusão de duplicatas do legado	279	2,95% do legado	`id_legado` de um cadastro só na ponte

Ficha	Decisão	Registros afetados	% da base	Informação perdida?
RDT-04	`CANCELADO` + motivo	10.510	12,82%	Motivo nulo em 3.161 do legado
RDT-05	Total = mercadoria + frete − desconto	82.000	100%	`valor_total` do e-commerce ≠ `total_amount` (frete)
RDT-06	`-1` → 0; frete negativo mantido	770 itens + 238 pedidos	0,97% dos itens do legado / 0,29% dos pedidos	Valor `-1` original só na `stg`
RDT-07	Mapeamento de categorias	97 de 600 produtos	16,17%	Sem categoria útil ou ambíguos
RDT-08	NULL para sentinela e data impossível	705	1,14% dos entregues	Data original só na `stg`
RDT-09	`America/Sao_Paulo`	7.095 pedidos mudam de dia	12,67% dos 56.000 do e-commerce	Hora do dia
RDT-10	Itens órfãos para quarentena	4.100 itens	1,48% dos 277.493	Valor desses itens fora do DW
RDT-11	Grão por `(sistema, id)` e reconciliação com a mesma definição	82.000 pedidos	100%	Pedidos de mesmo número nunca são fundidos
RDT-12	Sem cliente-sentinela; falha se aparecer	0	0%	Nada hoje
RDT-13	SAC e avaliações: cliente só pelo CPF, NULL onde não há par	1.229 sem cliente + 7.959 sem produto	6,47% do SAC / 24,87% das avaliações	Cliente do pedido não usado; hora e motivo livre
RDT-14	ABT com 15 atributos até o corte; alvo sem cancelados; prova por recálculo	22.106 clientes (4.801 recompram)	100% da base-cliente; taxa do alvo 21,7%	Status na extração, sem data de cancelamento; origem de CPF fora
RDT-15	Três vazamentos deliberados, medidos lado a lado	8.615 clientes com recência negativa	38,97% dos 22.106	Nada na DW; tabela com vazamento só para o experimento